

Mérési jegyzőkönyv

1. A mérés célja

A mérés célja a MergeSort és a QuickSort algoritmusok CPU-s és OpenCL-es megvalósításának összehasonlítása volt. A vizsgálat során azt elemeztem, hogy az egyes algoritmusok hogyan teljesítenek különböző elemszámok mellett, valamint hogy az OpenCL használata jelent-e gyorsulást kis méretű adathalmazok esetén.

2. Mérési környezet

A mérések Apple M1 alapú gépen készültek.

Az OpenCL-es futások során a program az Apple M1 eszközt használta.

A mérések ugyanazon a gépen, azonos környezetben történtek, így az eredmények közvetlenül összehasonlíthatók.

3. A mérés menete

A mérés során mind a MergeSort, mind a QuickSort esetén három különböző elemszámot vizsgáltam:

- 1000 elem
- 2000 elem
- 3000 elem

Minden elemszámnál 30 mintát futtattam. A CPU-s és az OpenCL-es verzió ugyanazokat a bemeneti adatokat kapta, ezért az összehasonlítás korrekt volt. Az OpenCL-es program minden futás után ellenőrizte, hogy a kapott eredmény rendezett-e, illetve megegyezik-e a CPU-s verzió eredményével. Az ellenőrzés minden esetben sikeres volt.

4. Összesített eredmények

MergeSort átlagos futási idők:

- 1000 elem: CPU = 0,070001 ms, OpenCL = 4,799736 ms
- 2000 elem: CPU = 0,149194 ms, OpenCL = 6,527957 ms
- 3000 elem: CPU = 0,222543 ms, OpenCL = 8,889767 ms

QuickSort átlagos futási idők:

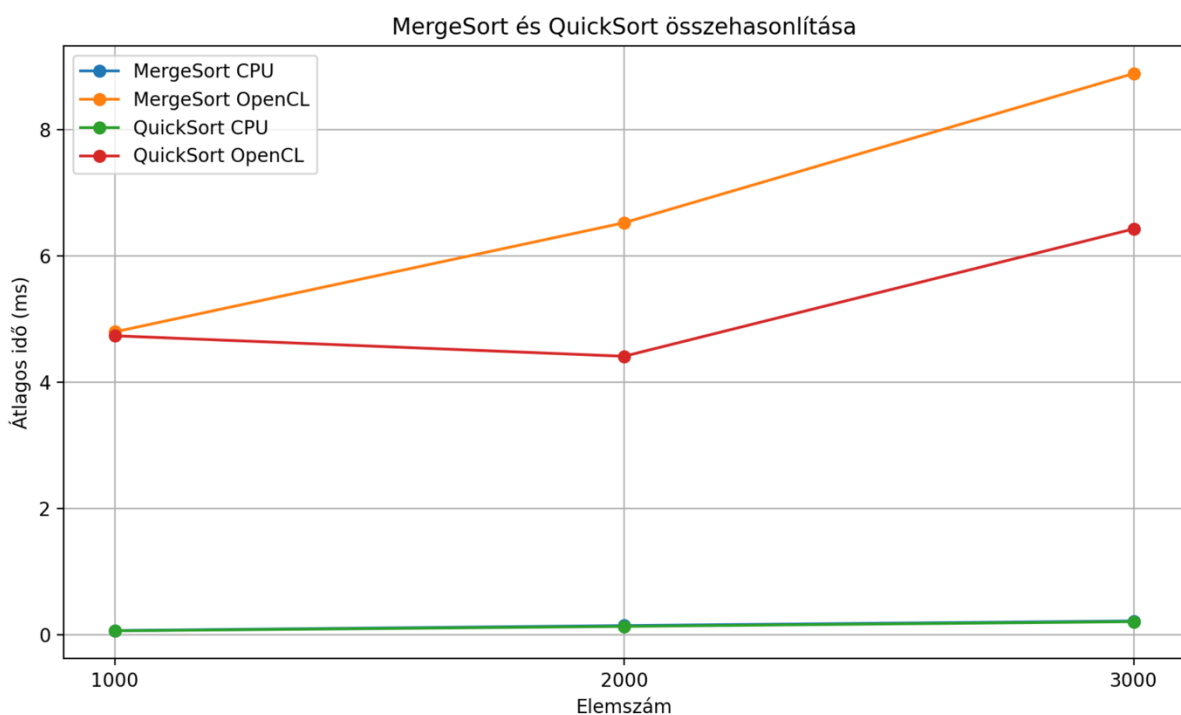
- 1000 elem: CPU = 0,066064 ms, OpenCL = 4,737565 ms
- 2000 elem: CPU = 0,136297 ms, OpenCL = 4,413472 ms
- 3000 elem: CPU = 0,211771 ms, OpenCL = 6,429890 ms

5. Következtetés

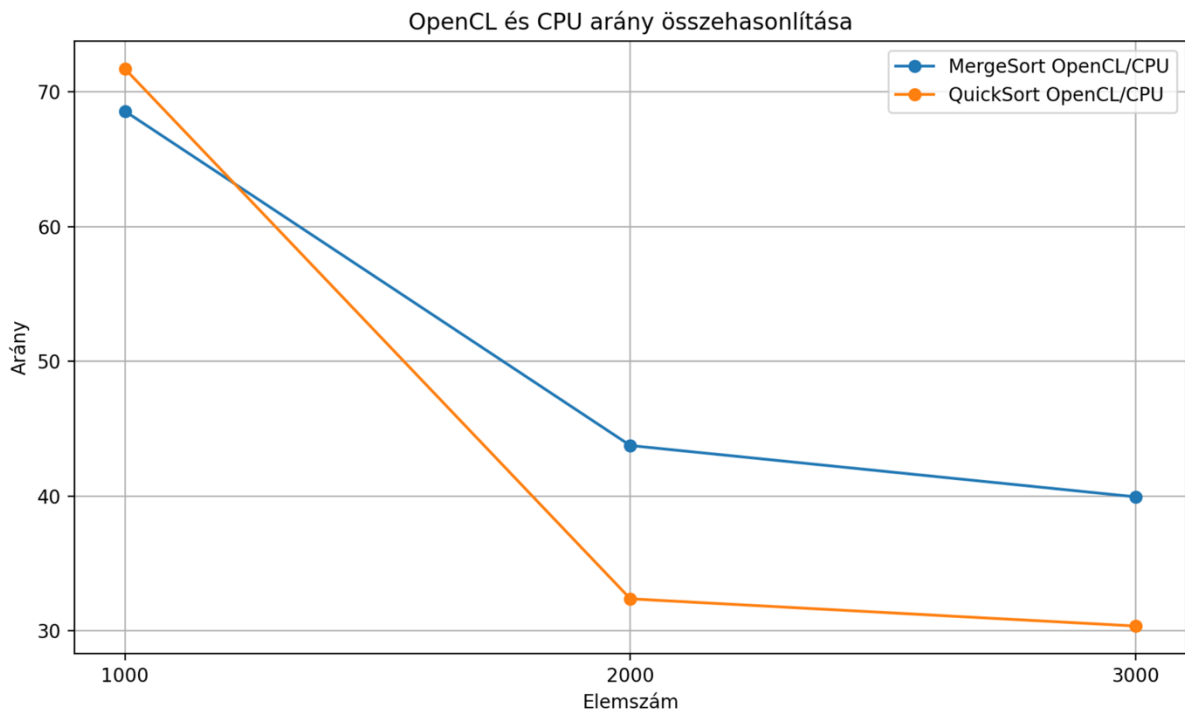
A mérés alapján megállapítható, hogy 1000, 2000 és 3000 elem rendezése esetén a CPU-s megoldás mind MergeSort, mind QuickSort esetén gyorsabb volt az OpenCL-es változatnál. A QuickSort CPU-s változata kis mértékben minden esetben jobb eredményt adott, mint a MergeSort CPU-s változata. OpenCL alatt a QuickSort szintén kedvezőbb eredményeket mutatott, különösen 2000 és 3000 elem esetén.

A legfontosabb következtetés az, hogy kis elemszámok mellett az OpenCL használata nem feltétlenül előnyös. A párhuzamos feldolgozás előnye csak nagyobb adathalmazoknál tudna jobban érvényesülni, amikor az OpenCL működésének többletköltsége már kisebb arányt képvisel a teljes futási időn belül.

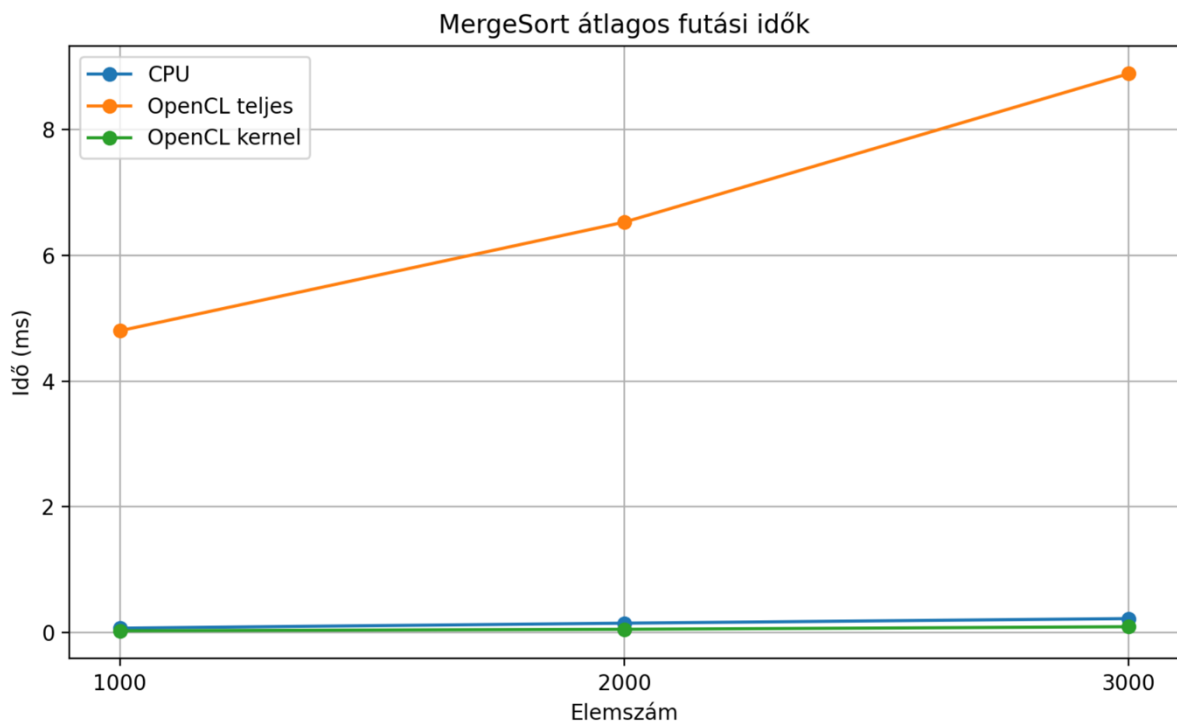
KÉPALÁÍRÁSOK



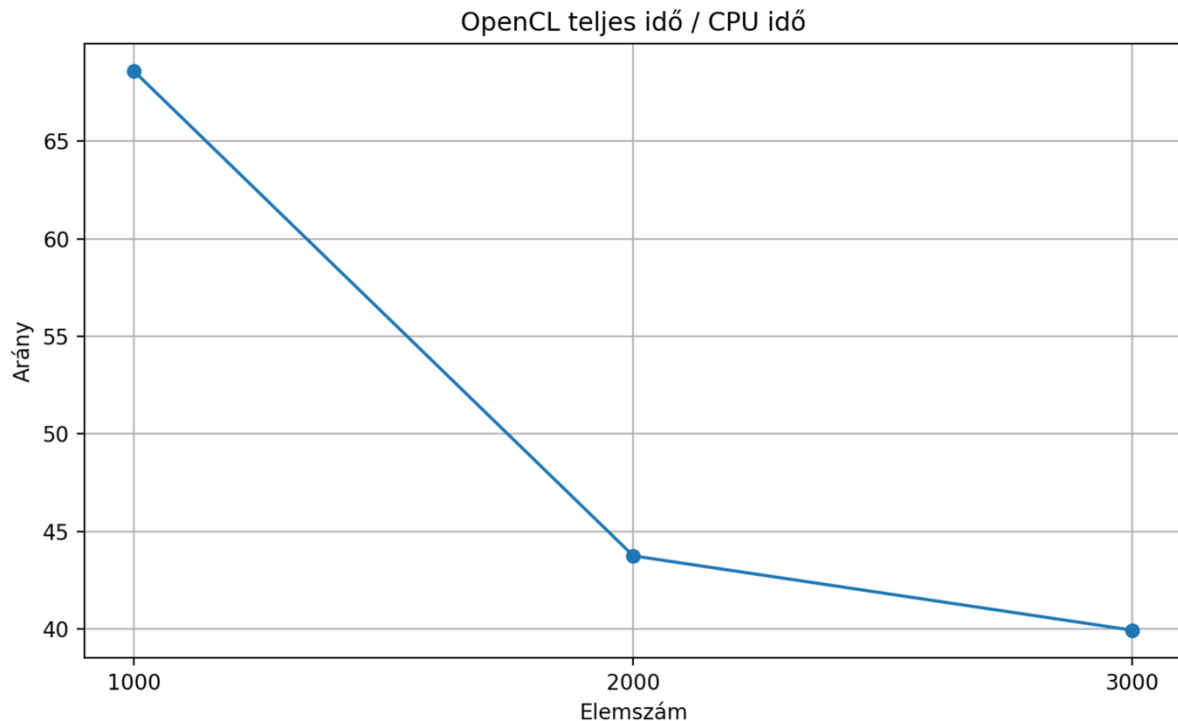
Az ábra alapján jól látható, hogy mindhárom vizsgált elemszámnál a CPU-s futás sokkal gyorsabb volt az OpenCL-es futásnál. Az is megfigyelhető, hogy CPU-n a QuickSort minden esetben kissé jobb eredményt adott, mint a MergeSort. OpenCL alatt szintén a QuickSort teljesített kedvezőbben, főleg 2000 és 3000 elem esetén. Az eredmények azt mutatják, hogy a vizsgált kis adatmennyiségnél az OpenCL többletköltsége nagyobb, mint a párhuzamos végrehajtásból származó előny.



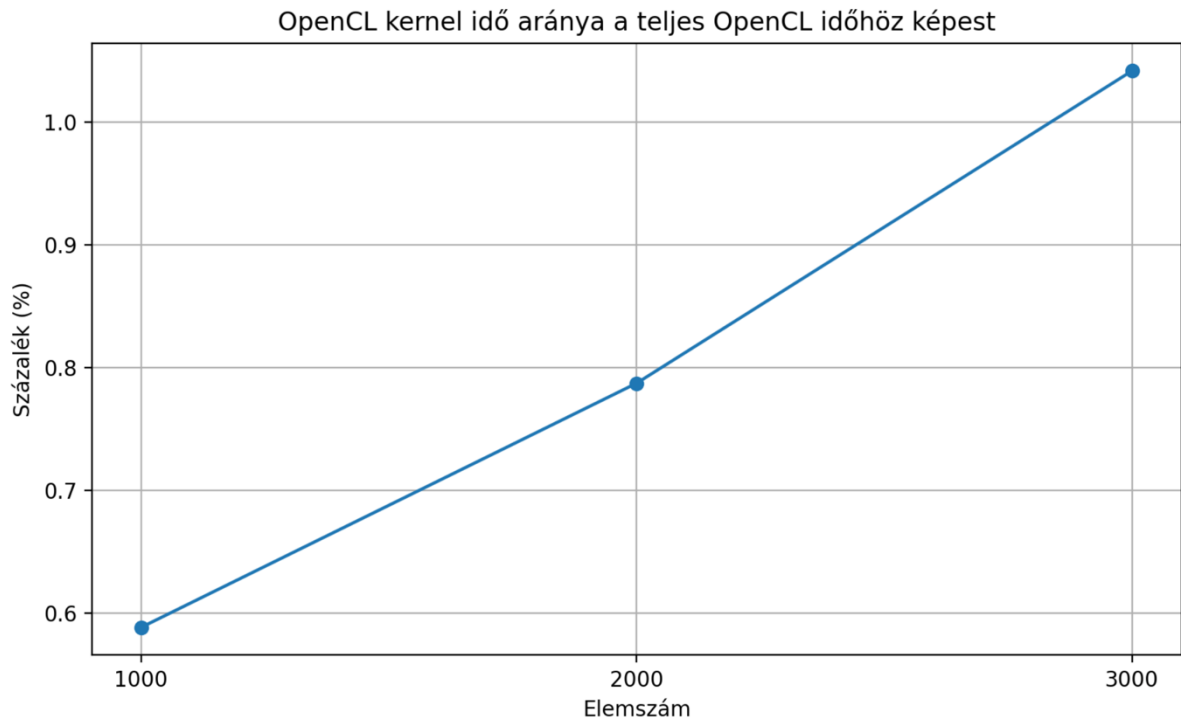
Az ábra azt mutatja meg, hogy az OpenCL teljes futási ideje hányszorosa a CPU-s futási időnek. Mindkét algoritmusnál nagy arányok láthatók, tehát ebben a mérési tartományban a CPU egyértelműen hatékonyabb volt. Az is látszik, hogy a QuickSort OpenCL/CPU aránya 2000 és 3000 elemnél kedvezőbb, mint a MergeSorté, vagyis az OpenCL-es QuickSort viszonylag jobb eredményt adott. Ettől függetlenül egyik OpenCL-megoldás sem volt gyorsabb a CPU-nál.



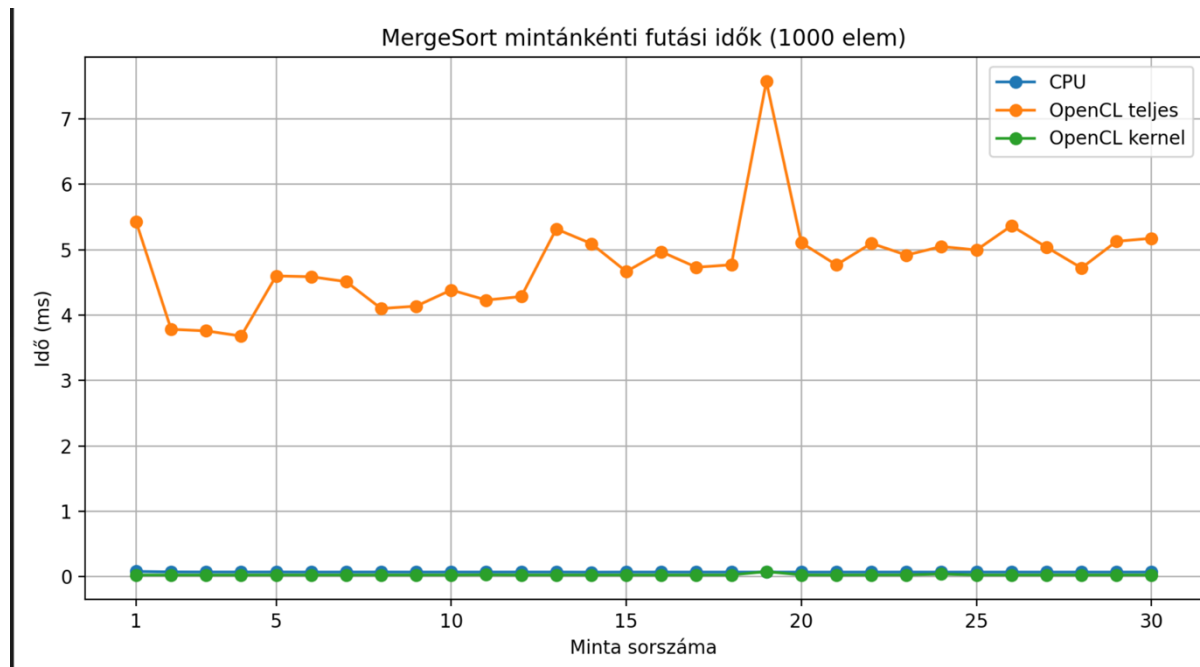
Az ábrán a MergeSort CPU-s, OpenCL teljes és OpenCL kernel futási idejének átlaga látható. Jól megfigyelhető, hogy a CPU-s megvalósítás minden elemszámnál nagyságrendekkel gyorsabb volt. Az OpenCL kernelidő önmagában kicsi, a teljes OpenCL futási időt főként a járulékos költségek növelik meg.



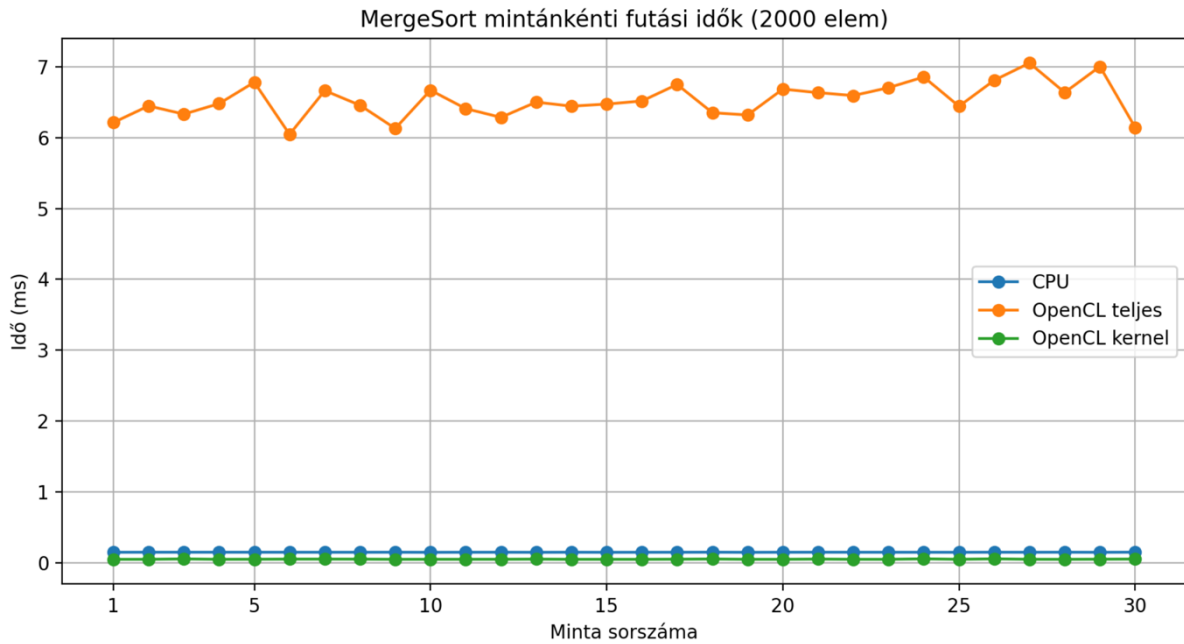
Ez az ábra azt mutatja, hogy a MergeSort OpenCL-es teljes futási ideje hányszorosa a CPU-s futási időnek. Az arány minden esetben nagy, ami azt jelzi, hogy a vizsgált elemszámok mellett a CPU lényegesen hatékonyabb. Az elemszám növekedésével az arány csökken, tehát nagyobb adatmennyiségnél az OpenCL relatív hátránya mérséklődik.



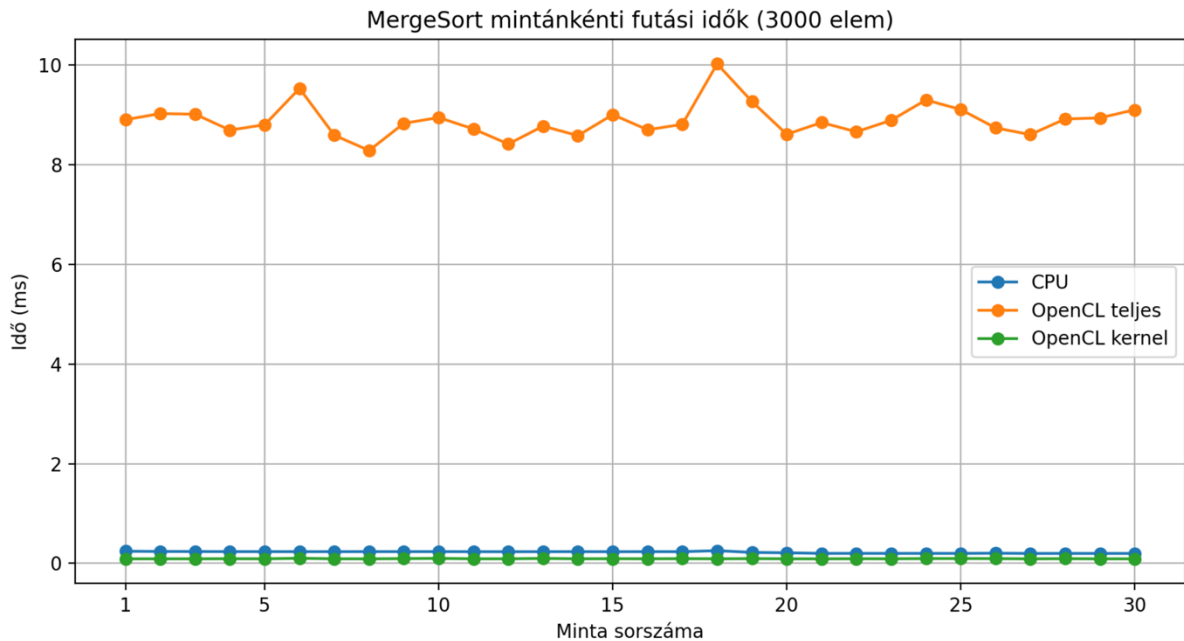
Az ábrából az látszik, hogy maga a kernelidő csak kis részét adja a teljes OpenCL futási időnek. Ez azt mutatja, hogy a teljes végrehajtási időt nem maga a rendezés dominálja, hanem az OpenCL környezet kezelésének többletköltsége. Emiatt kis elemszámnál az OpenCL nem tudott előnyt mutatni.



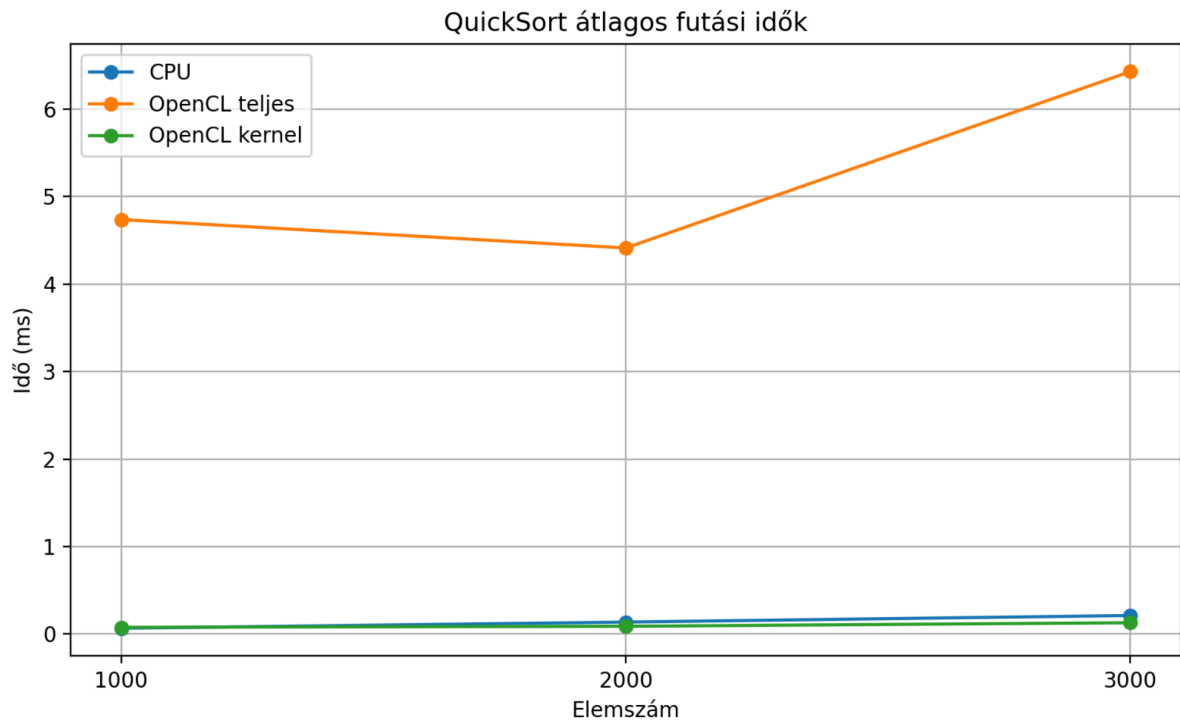
Az ábra a 30 külön mérés eredményét mutatja 1000 elemre. A CPU-s futások stabilan és nagyon kis idővel zajlottak, míg az OpenCL teljes ideje jelentősen nagyobb volt. A kernelidő itt is alacsony maradt a teljes OpenCL időhöz viszonyítva.



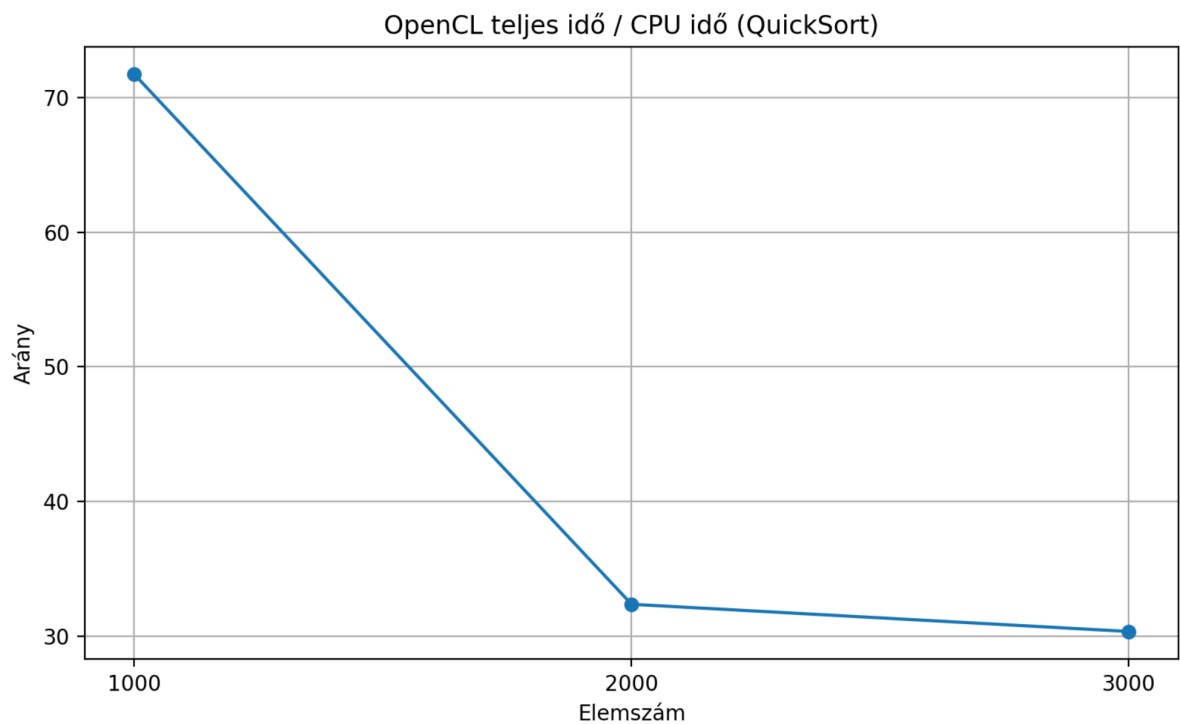
2000 elemnél a CPU-s mérések továbbra is stabilak maradtak. Az OpenCL teljes futási ideje nőtt, azonban a kernelidő növekedése ehhez képest jóval kisebb mértékű volt. Ez megerősíti, hogy a teljes idő jelentős részét az OpenCL overhead adja.



3000 elemnél a CPU és az OpenCL közötti különbség továbbra is egyértelműen megmaradt. A teljes OpenCL idő emelkedett, a kernelidő viszont továbbra is a teljes idő kis hányada maradt. Az eredmények alapján a MergeSort OpenCL-változata ebben a tartományban nem volt versenyképes a CPU-s megoldással.

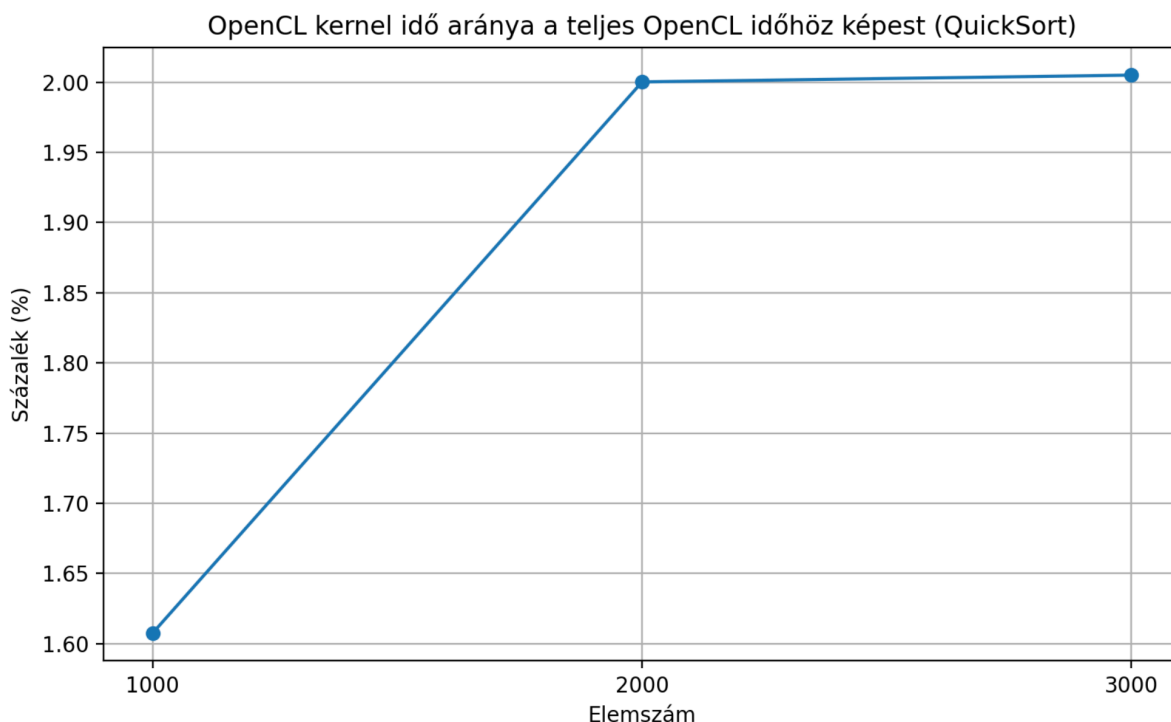


Az ábra a QuickSort CPU-s, OpenCL teljes és OpenCL kernel futási idejének átlagát mutatja. A CPU-s változat itt is minden elemszámnál gyorsabb volt. Ugyanakkor az OpenCL teljes futási ideje a MergeSort OpenCL-eredményeihez képest kedvezőbbnek bizonyult, különösen 2000 és 3000 elem esetén.

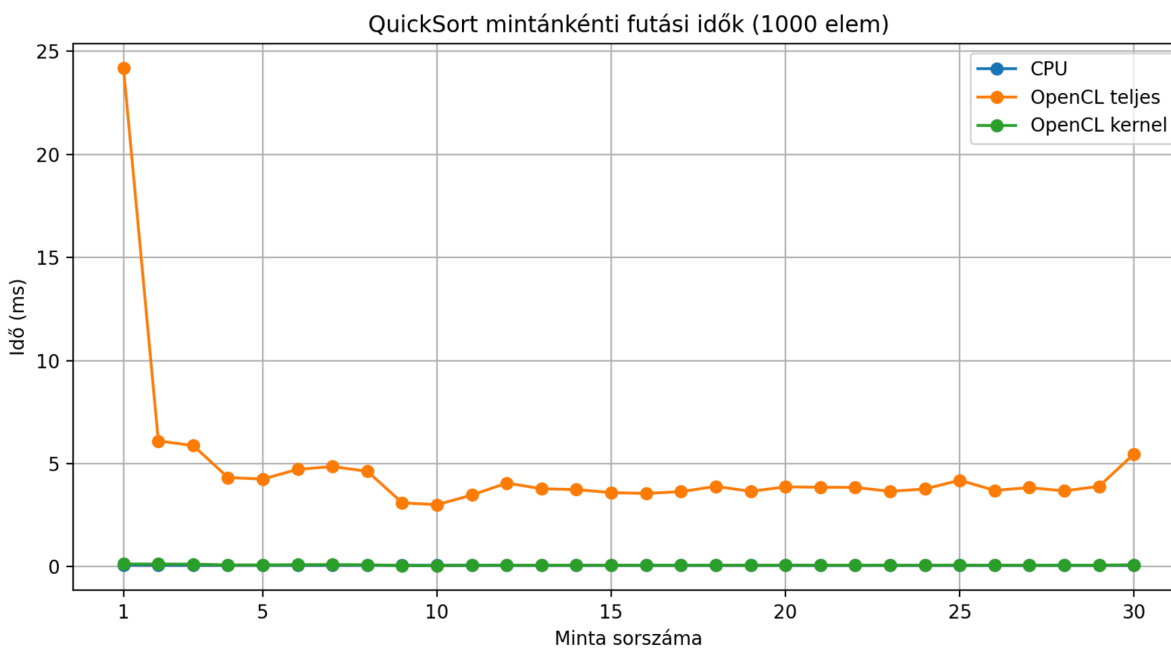


Az ábra megmutatja, hogy a QuickSort OpenCL-es teljes futási ideje hányszorosa a CPU-s futási időnek. Bár az OpenCL itt sem tudta megelőzni a CPU-t, az arány nagyobb

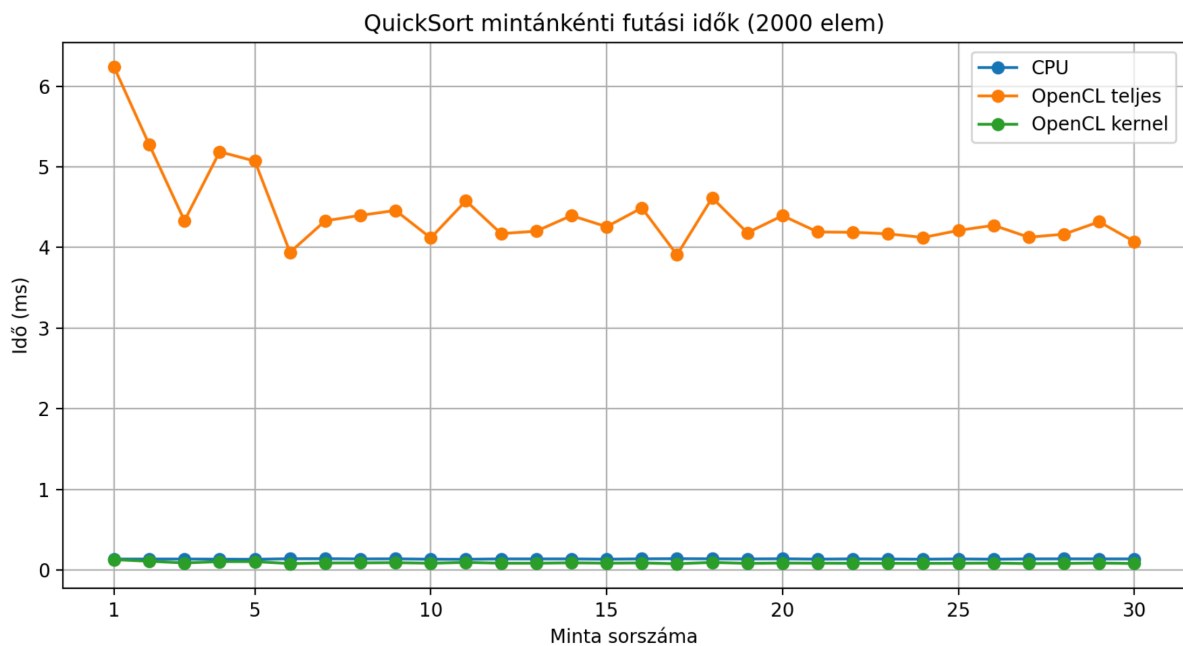
elemszámoknál kedvezőbb, mint a MergeSort esetében. Ez arra utal, hogy a QuickSort OpenCL-változata ebben a mérési tartományban hatékonyabb volt a MergeSort OpenCL-változatánál.



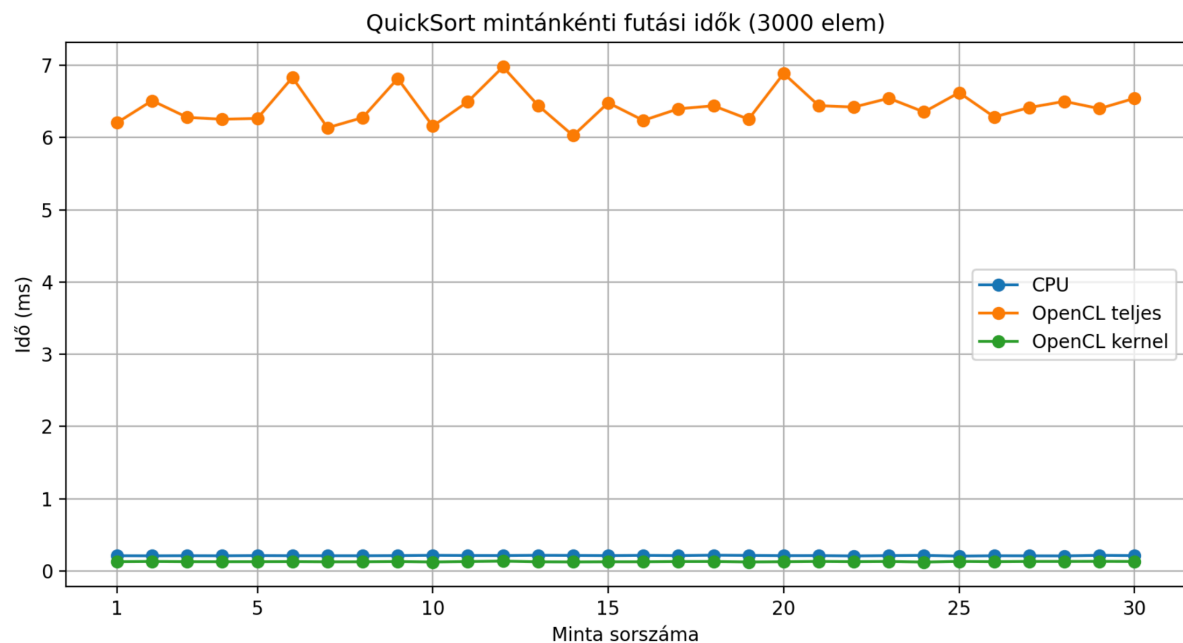
Az ábra azt szemlélteti, hogy a QuickSort esetén is a kernelidő csak kis részét teszi ki a teljes OpenCL futási időnek. A teljes idő nagy részét itt sem maga a rendezési művelet, hanem az OpenCL használatával járó többletköltség adja. Ez megmagyarázza, hogy miért maradt a CPU-s megoldás gyorsabb.



1000 elem esetén a CPU-s futási idők alacsonyok és stabilak. Az OpenCL teljes ideje jóval magasabb, és az első mérésnél egy kiugró érték is megfigyelhető. Ez tipikusan a futtatási környezet inicializációjával és a rendszerterheléssel magyarázható.



2000 elemnél a QuickSort CPU-s és OpenCL-es futásai is stabilabb képet mutatnak. Az OpenCL teljes idő még mindig magasabb a CPU-időnél, viszont kedvezőbb, mint a MergeSort OpenCL-eredménye. Ez arra utal, hogy ugyanazon a hardveren a QuickSort OpenCL-megvalósítása hatékonyabban viselkedett.



3000 elemnél a CPU-s QuickSort továbbra is gyors maradt, az OpenCL teljes ideje pedig mérsékeltebben növekedett, mint a MergeSort esetében. Ez összhangban van az összegző

grafikonokkal, ahol a QuickSort OpenCL-változata kedvezőbb eredményt adott. A CPU előnye azonban itt is egyértelmű maradt.

RÖVID ZÁRÓ KÖVETKEZTETÉS

A mérések alapján megállapítható, hogy 1000, 2000 és 3000 elem esetén a CPU-s rendezések gyorsabbak voltak az OpenCL-es megoldásoknál. Ennek oka, hogy ilyen kis adatmennyiségnél az OpenCL működtetésének többletköltsége nagyobb, mint a párhuzamos végrehajtásból származó előny. A CPU-s eredmények közül a QuickSort minden esetben kissé gyorsabb volt a MergeSortnál, és OpenCL alatt is a QuickSort adott kedvezőbb eredményeket. A vizsgálatból az a következtetés vonható le, hogy kis elemszámok esetén a CPU-s megoldás célszerűbb, míg az OpenCL előnye várhatóan csak nagyobb bemeneteknél jelenne meg.